

# Sorting/searching at scale

## COMPLETE LEARNING GUIDE

### What You'll Learn

Sorting and searching are the most fundamental operations in computer science, and while calling `Arrays.sort()` is one line of code, understanding what happens underneath — which algorithm runs, why, and what its actual guarantees are — is essential both for interview questions and for reasoning about performance at real scale. This guide covers the two dominant comparison-based sorting algorithms (quicksort and mergesort) with their complexity and stability trade-offs, exactly which algorithm Java's `Arrays.sort()` and `Collections.sort()` actually run for primitives versus objects, binary search and its common variants (lower bound, upper bound), practical guidance on choosing a sort at scale including a brief note on external sorting for data too large to fit in memory, and the rotated-sorted-array problem as a staple binary-search variant interviewers return to constantly.

## 1. Quicksort: Partition-Based, In-Place, Usually Fastest in Practice

Quicksort picks a pivot element, partitions the array so everything smaller than the pivot comes before it and everything larger comes after (the pivot ends up in its final sorted position after partitioning), then recursively sorts the two partitions. Average case is  $O(n \log n)$  with excellent constant factors and in-place  $O(\log n)$  auxiliary space (just the recursive call stack), but worst case is  $O(n^2)$  when partitioning is consistently unbalanced — classically triggered by an already-sorted array with a naive "always pick the first element" pivot strategy, which is exactly why production implementations use randomized or median-of-three pivot selection to make the worst case practically unreachable.

### JAVA

```
// Quicksort -- average  $O(n \log n)$ , worst case  $O(n^2)$ ,  $O(\log n)$  auxiliary space (in-place)
public static void quicksort(int[] arr, int low, int high) {
    if (low < high) {
        int pivotIndex = partition(arr, low, high);
        quicksort(arr, low, pivotIndex - 1);
        quicksort(arr, pivotIndex + 1, high);
    }
}

private static int partition(int[] arr, int low, int high) {
    int pivot = arr[high]; // choosing the last element as pivot
    // (simple, not worst-case-safe) // boundary of the "smaller than pivot" region
    for (int j = low; j < high; j++) {
        if (arr[j] < pivot) {
```



```

        int temp = arr[i]; arr[i] = arr[j]; arr[j] = temp;
    }
}
int temp = arr[i + 1]; arr[i + 1] = arr[high]; arr[high] = temp;    // place
pivot in final position
return i;
}

```

## 2. Mergesort: Divide-and-Conquer, Stable, Guaranteed $O(n \log n)$

Mergesort recursively splits the array in half until subarrays of size 1 remain (trivially sorted), then merges sorted subarrays back together in linear time per merge step — this gives a guaranteed  $O(n \log n)$  in every case, including worst case, unlike quicksort's  $O(n^2)$  worst case. The trade-off is  $O(n)$  auxiliary space for the merge step's temporary array, versus quicksort's  $O(\log n)$ , and mergesort's real-world constant factors are typically somewhat worse than quicksort's despite the same asymptotic complexity. Mergesort is also stable — equal elements retain their original relative order — a property quicksort's typical in-place partitioning does not guarantee, and stability matters whenever sorting by one key should preserve a previous sort by another key.

## 3. What Java's `Arrays.sort()` Actually Runs

Java's `Arrays.sort()` uses different algorithms depending on the element type, a detail interviewers specifically like to probe: for arrays of primitives (`int[]`, `double[]`, etc.), it uses a Dual-Pivot Quicksort, chosen for its excellent average-case performance and in-place operation, accepting that primitives have no notion of "identity" so stability is irrelevant. For arrays of objects (or any `List` via `Collections.sort()`), it uses a variant of TimSort (a hybrid of merge sort and insertion sort, originally developed for Python), chosen specifically because it's stable — object sorts frequently need to preserve relative order of equal elements — and TimSort is also adaptive, running faster than  $O(n \log n)$  on data that's already partially sorted, a common real-world pattern.

- `Arrays.sort(int[])` / other primitive arrays — Dual-Pivot Quicksort,  $O(n \log n)$  average, not stable (irrelevant for primitives).
- `Arrays.sort(Object[])` / `Collections.sort(List<T>)` — TimSort,  $O(n \log n)$  worst case, stable, adaptive on nearly-sorted input.
- Stability matters whenever a later sort must preserve an earlier sort's relative ordering of equal keys (e.g. sort by last name, then by department, expecting last-name order preserved within each department).

## 4. Binary Search and Its Variants

Binary search finds a target in a sorted array in  $O(\log n)$  by repeatedly halving the search space, comparing the target against the middle element and discarding the half that can't contain it. Beyond simple existence checking, the lower bound variant finds the first index where a value  $\geq$  target could be inserted while keeping the array sorted (the leftmost valid insertion point), and the upper bound variant finds the first index where a value  $>$  target could be inserted (the rightmost valid insertion point) — together, `upperBound` - `lowerBound` gives the count of elements equal to target in  $O(\log n)$ , without a linear scan.

#### JAVA

```
// Lower bound: first index where arr[index] >= target -- O(log n)
public static int lowerBound(int[] arr, int target) {
    int low = 0, high = arr.length;    // note: high starts at length, not length
    - 1 while (low < high) {
        int mid = low + (high - low) / 2;    // avoids integer overflow vs. (low
    + high) / 2
        if (arr[mid] < target) low = mid + 1;
        else high = mid;
    }
    return low;
}
```

## 5. Searching a Rotated Sorted Array

A sorted array rotated at an unknown pivot (e.g. [4,5,6,7,0,1,2]) still supports  $O(\log n)$  search, but the standard binary search comparison isn't directly enough — at each step, at least one of the two halves (left of mid, or right of mid) is guaranteed to still be normally sorted, so the algorithm first determines which half is sorted (by comparing the endpoints), then checks whether the target falls within that sorted half's range to decide which side to recurse into, falling back to the other half otherwise. This is one of the most commonly repeated "sorted array with a twist" interview questions precisely because it tests whether you can adapt binary search's invariant rather than just recite the textbook version.

#### JAVA

```
// Search in a rotated sorted array -- O(log n)
public static int searchRotated(int[] arr, int target) {
    int low = 0, high = arr.length - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (arr[mid] == target) return mid;

        if (arr[low] <= arr[mid]) {           // left half [low..mid]
            // is normally sorted
            if (arr[low] <= target && target < arr[mid]) high = mid - 1;
            else low = mid + 1;
        } else {                             // right half
            // [mid..high] is normally sorted
            if (arr[mid] < target && target <= arr[high]) low = mid + 1;
            else high = mid - 1;
        }
    }
    return -1;
}
```

## 6. Sorting "At Scale": A Brief Note on External Sorting

When data to be sorted is too large to fit in main memory — a common "at scale" scenario — external sorting (typically external mergesort) processes it in chunks: read a chunk that fits in memory, sort it in-place with a standard in-memory algorithm, write the sorted chunk back to disk, and once all chunks are sorted, merge them together using a k-way merge (often backed by a priority queue holding one element from each chunk at a time) that reads only small buffered portions of each sorted chunk at once. This is conceptually the same divide-and-conquer idea as mergesort, just with "divide" driven by memory constraints rather

than a fixed recursive halving — worth knowing the name and general shape of this technique even if you're not expected to implement it in a typical interview.

## Common Pitfalls

---

- Assuming `Arrays.sort()` always uses the same algorithm regardless of type — it's Dual-Pivot Quicksort for primitives but TimSort for objects, and the distinction matters for stability.
- Using an in-place quicksort when stability is actually required (equal elements must preserve relative order), when mergesort or TimSort should be used instead.
- Applying plain binary search directly to a rotated sorted array without first determining which half is still normally sorted at each step.
- Computing mid as  $(low + high) / 2$  instead of  $low + (high - low) / 2$ , risking integer overflow on very large arrays.
- Confusing lower bound and upper bound semantics, off-by-one on which index each actually returns.
- Forgetting binary search requires the array to already be sorted — running it on unsorted data silently produces an incorrect (and not even consistently wrong) result.
- Not knowing quicksort's worst case ( $O(n^2)$ , triggered by consistently unbalanced partitioning) when asked to compare it against mergesort's guaranteed  $O(n \log n)$ .

## Key Takeaways

---

- Quicksort:  $O(n \log n)$  average,  $O(n^2)$  worst case,  $O(\log n)$  space, in-place, not stable — typically fastest in practice.
- Mergesort:  $O(n \log n)$  guaranteed in every case,  $O(n)$  space, stable — the safer choice when worst-case guarantees or stability matter.
- Java uses Dual-Pivot Quicksort for primitive arrays and a stable, adaptive TimSort for object arrays and `Collections.sort()` — know which runs where.
- Binary search variants (lower bound, upper bound) locate insertion points, not just existence, and together give an  $O(\log n)$  count of equal elements.
- A rotated sorted array is still searchable in  $O(\log n)$  by determining which half remains normally sorted at each step.
- External sorting (chunk, sort, k-way merge) extends the mergesort idea to data too large to fit in memory.

## Learn More

**GeeksforGeeks — Sorting Algorithms**

<https://www.geeksforgeeks.org/dsa/sorting-algorithms/>

**GeeksforGeeks — Search in Rotated Sorted Array**

<https://www.geeksforgeeks.org/dsa/search-an-element-in-a-sorted-and-pivoted-array/>

**Baeldung — Java's Arrays.sort() and TimSort**

<https://www.baeldung.com/java-arrays-sort>